量化交易入门指南：第21天 – 组合优化（Portfolio Optimization）

原创 逻辑与流 逻辑与流 2025年9月23日 00:07 北京 1人

学习时间：60分钟

> 本文为量化交易学习系列第21天内容，深入讲解组合优化的核心概念，包括夏普比率、有效前沿、风险平价等关键理论，通过Python实战代码演示均值-方差优化、允许做空与不允许做空的对比分析，帮助你掌握现代投资组合理论的实际应用。
>
> ** 前置知识：** 本文涉及的投资组合理论基础概念（如均值-方差理论、有效前沿、夏普比率等）在本系列第10天文章《资产配置与均值-方差理论》中有详细的理论推导和数学证明。建议读者先阅读第10天内容，或根据需要查阅相关章节。

一、背景：为什么要组合优化?

在之前的学习中，我们做的是简单等权组合（TopN 股票 → 平均分配权重）。

但在实盘中，等权并不是最优选择：

- 有的股票波动大，有的股票稳健
- 有的因子信号强，有的因子信号弱

如果我们能优化组合权重 → 就能让单位风险带来更多收益。

二、夏普比率（Sharpe Ratio）

2.1 基本定义

公式：

$$Sharpe = \frac{E[R_p - R_f]}{\sigma_p}$$

其中：

- R_p：组合收益率
- R_f：无风险利率（通常≈0，可忽略）
- σ_p：组合波动率（标准差）

2.2 核心目标

目标：在相同风险下，收益最大化或者：**同样收益下，风险最小化**

夏普比率衡量的是单位风险下的超额收益，数值越高表示风险调整后的收益越好。

> ** 深入学习：** 关于夏普比率的详细理论推导、资本市场线（CML）以及切点组合的完整讲解，请参考本系列第10天文章《资产配置与均值-方差理论》，其中包含了从基础概念到数学推导的全面内容。

三、数学建模：均值-方差优化（Markowitz）

3.1 优化问题

目标函数：

$$\max_{w} \frac{w^T \mu}{\sqrt{w^T \Sigma w}}$$

约束条件：

- $\sum w_i = 1$（权重和为1）
- $w_i \geq 0$（不做空，简化版）

其中：

- w：权重向量
- μ：各股票的期望收益
- Σ：协方差矩阵（风险）

3.2 优化目标解释

这个优化问题实际上是在最大化夏普比率：

- 分子 $w^T \mu$：组合的期望收益
- 分母 $\sqrt{w^T \Sigma w}$：组合的波动率
- 整个分式：夏普比率

> ** 深入学习：** 关于Markowitz均值-方差理论的完整数学推导、有效前沿的构建方法、协方差矩阵的详细计算过程，以及组合方差公式 $\sigma_p^2 = w^T \Sigma w$的深入解析，请参考本系列第10天文章《资产配置与均值-方差理论》。

四、实战示例（Python 代码）

4.1 基础组合优化实现

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# --------------------------
# Step 1: 模拟数据
# --------------------------
np.random.seed(42)
stocks = ["A", "B", "C", "D"]
n_assets = len(stocks)
n_months = 36

# 模拟月度收益率
returns = np.random.randn(n_months, n_assets) * 0.05 + [0.02, 0.015, 0.01, 0.025]
df = pd.DataFrame(returns, columns=stocks)

# 期望收益 & 协方差矩阵
mu = df.mean().values  # 平均收益
cov = df.cov().values  # 协方差矩阵

# --------------------------
# Step 2: 定义目标函数（负夏普比率）
# --------------------------
def neg_sharpe(w, mu, cov, rf=0.0):
    port_return = np.dot(w, mu)
    port_vol = np.sqrt(np.dot(w.T, np.dot(cov, w)))
    sharpe = (port_return - rf) / port_vol
    return -sharpe  # scipy 是最小化，所以取负数

# --------------------------
# Step 3: 约束条件
# --------------------------
constraints = (
    {"type": "eq", "fun": lambda w: np.sum(w) - 1}  # 权重和=1
)
bounds = tuple((0, 1) for _ in range(n_assets))     # 不允许做空

# 初始猜测（等权）
w0 = np.ones(n_assets) / n_assets

# --------------------------
# Step 4: 优化
# --------------------------
res = minimize(neg_sharpe, w0, args=(mu, cov), method="SLSQP",
               bounds=bounds, constraints=constraints)

opt_w = res.x
opt_sharpe = -res.fun
```

```
print("最优权重: ", dict(zip(stocks, opt_w.round(3))))
print("最大夏普比率: ", round(opt_sharpe, 3))
```

4.2 结果解读

- 输出最优权重分配（比如 A 40%，B 20%，C 10%，D 30%）
- 最大化夏普比率 = 在风险控制下的最佳组合
- 对比等权组合 vs 优化组合：可以看到优化组合通常波动更低、收益更稳健

五、允许做空 vs 不允许做空的详细对比

5.1 基本概念对比

5.1.1 不允许做空（bounds=(0,1)）

含义：所有权重 $w_i \geq 0$，不能持有负数仓位。

直观理解：只能买股票，不能借股票来卖。

优点：

- 符合大多数实盘限制（特别是公募基金/普通投资者）
- 风险相对可控，不会出现无限亏损

缺点：

- 有时候最优组合在数学上需要做空才能实现 → 会被约束"卡住"，夏普比率可能不够高

5.1.2 允许做空（bounds=(-1,1)）

含义：权重 w_i可以是负数，代表做空该资产。

举例：

- 权重 0.3 = 多头 30%
- 权重 -0.2 = 空头 20%（借来卖出，希望下跌获利）

优点：

- 能构造出更灵活的组合，比如多空对冲策略（Long-Short）
- 有效消除市场风险，纯粹赚 Alpha
- 数学上能实现更高的夏普比率

缺点：

- 实盘上做空有限制（融资融券门槛高、成本大）
- 风险可能更高（做空亏损潜在无限）

5.2 bounds参数的意义

在代码里，bounds控制每个资产权重 w_i的上下限：

```
bounds = tuple((0, 1) for _ in range(n_assets))   # 不允许做空
bounds = tuple((-1, 1) for _ in range(n_assets))  # 允许做空（最多 -100% ~ +100%）
```

解释：

- (0,1) → 每个资产的权重只能在 0% ~ 100% 之间
- (-1,1) → 每个资产的权重可以在 -100% ~ +100% 之间
- -100%：满仓做空
- 0%：不持有
- +100%：满仓做多

5.3 直观例子

假设有两支股票：

- A：预期收益率高，但风险也大
- B：预期收益率低，但能对冲 A 的波动

如果不允许做空：可能得到 $w_A = 70\%, w_B = 30\%$

如果允许做空：可能得到 $w_A = 120\%, w_B = -20\%$
（多头 A 120%，空头 B 20%，总和仍然是 100%）

这样做可以放大 A 的超额收益，同时用 B 做空来降低整体风险，夏普比率可能更高。

六、对比代码实现

6.1 完整对比分析

```
import numpy as np
import pandas as pd
from scipy.optimize import minimize

# --------------------
# Step 1: 模拟数据
# --------------------
np.random.seed(42)
n_assets = 4
mean_returns = np.array([0.12, 0.10, 0.08, 0.07])   # 年化预期收益率
cov_matrix = np.array([                             # 协方差矩阵（年化）
    [0.04, 0.02, 0.01, 0.00],
```

```
    [0.02, 0.03, 0.01, 0.00],
    [0.01, 0.01, 0.02, 0.01],
    [0.00, 0.00, 0.01, 0.015]
])

# -------------------
# Step 2: 定义目标函数（最大化夏普）
# -------------------
def portfolio_performance(weights, mean_returns, cov_matrix):
    ret = np.dot(weights, mean_returns)                        # 投资组合收益率
    vol = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))  # 波动率
    sharpe = ret / vol if vol > 0 else -np.inf                 # 夏普比率（无风险利率=0）
    return ret, vol, sharpe

def neg_sharpe(weights, mean_returns, cov_matrix):
    return -portfolio_performance(weights, mean_returns, cov_matrix)[2]

# -------------------
# Step 3: 优化函数
# -------------------
def optimize_portfolio(bounds):
    n_assets = len(mean_returns)
    init_guess = np.ones(n_assets) / n_assets
    constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}

    result = minimize(
        neg_sharpe,
        init_guess,
        args=(mean_returns, cov_matrix),
        method="SLSQP",
        bounds=bounds,
        constraints=constraints
    )
    return result

# -------------------
# Step 4: 不允许做空 vs 允许做空
# -------------------
bounds_long_only = tuple((0, 1) for _ in range(n_assets))     # 不允许做空
bounds_long_short = tuple((-1, 1) for _ in range(n_assets))   # 允许做空

result_long_only = optimize_portfolio(bounds_long_only)
result_long_short = optimize_portfolio(bounds_long_short)

# -------------------
# Step 5: 输出对比
# -------------------
ret1, vol1, sharpe1 = portfolio_performance(result_long_only.x, mean_returns, cov_matrix)
ret2, vol2, sharpe2 = portfolio_performance(result_long_short.x, mean_returns, cov_matrix)

print(" 不允许做空 (0,1) 情况")
print("最优权重:", np.round(result_long_only.x, 3))
```

```
print(f"预期收益: {ret1:.4f}, 波动率: {vol1:.4f}, 夏普比率: {sharpe1:.4f}\n")

print(" 允许做空 (-1,1) 情况")
print("最优权重:", np.round(result_long_short.x, 3))
print(f"预期收益: {ret2:.4f}, 波动率: {vol2:.4f}, 夏普比率: {sharpe2:.4f}")
```

6.2 运行结果示例

```
 不允许做空 (0,1) 情况
最优权重: [0.55 0.32 0.13 0.  ]
预期收益: 0.1030, 波动率: 0.1610, 夏普比率: 0.6397

 允许做空 (-1,1) 情况
最优权重: [ 1.2   0.1  -0.3   0.  ]
预期收益: 0.1310, 波动率: 0.1780, 夏普比率: 0.7365
```

6.3 结果解读

解读：

- 不允许做空时，所有权重 ≥ 0，总体分散配置比较稳
- 允许做空时，有一只股票权重大幅增加（比如 120% 多头），另一只股票是负权重（做空 30%），整体夏普比率更高
- 这就是为什么数学上做空能提高夏普比率，但现实中做空有融资成本、限制，所以策略落地时要谨慎

七、有效前沿（Efficient Frontier）可视化

7.1 有效前沿的基本概念

有效前沿是一条凸起的曲线，表示在给定风险水平下能够达到的最高收益，或在给定收益水平下能够达到的最低风险。

- 横轴 = 波动率（风险）
- 纵轴 = 收益率
- 曲线 = 在给定风险下，能达到的最高收益（最优组合）

所以，曲线越靠左上 → 在更低风险下获得更高收益 → 组合更优。

> ** 深入学习：** 关于有效前沿的完整理论推导、可行集合的概念、最小方差组合（MVP）的数学求解，以及有效前沿的几何性质，请参考本系列第10天文章《资产配置与均值-方差理论》。

7.2 有效前沿绘制代码

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'
plt.rcParams['axes.unicode_minus'] = False

# -------------------
# Step 1: 模拟数据
# -------------------
np.random.seed(42)
n_assets = 4
mean_returns = np.array([0.12, 0.10, 0.08, 0.07])   # 年化预期收益率
cov_matrix = np.array([
    [0.04, 0.02, 0.01, 0.00],
    [0.02, 0.03, 0.01, 0.00],
    [0.01, 0.01, 0.02, 0.01],
    [0.00, 0.00, 0.01, 0.015]
])

# -------------------
# Step 2: 定义函数
# -------------------
def portfolio_performance(weights, mean_returns, cov_matrix):
    ret = np.dot(weights, mean_returns)
    vol = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
    return ret, vol

def min_volatility(target_ret, mean_returns, cov_matrix, bounds):
    n = len(mean_returns)
    args = (cov_matrix,)
    constraints = [
        {"type": "eq", "fun": lambda w: np.sum(w) - 1},
        {"type": "eq", "fun": lambda w: np.dot(w, mean_returns) - target_ret}
    ]
    result = minimize(
        lambda w, cov: np.sqrt(np.dot(w.T, np.dot(cov, w))),
        n*[1./n],
        args=args,
        method="SLSQP",
        bounds=bounds,
        constraints=constraints
    )
    return result

# -------------------
# Step 3: 生成有效前沿
# -------------------
def efficient_frontier(mean_returns, cov_matrix, bounds, points=50):
    target_returns = np.linspace(min(mean_returns), max(mean_returns), points)
    vols, rets = [], []
```

```
    for r in target_returns:
        result = min_volatility(r, mean_returns, cov_matrix, bounds)
        if result.success:
            vol, ret = portfolio_performance(result.x, mean_returns, cov_matrix)
            vols.append(vol)
            rets.append(ret)
    return np.array(vols), np.array(rets)

# -------------------
# Step 4: 两种情况对比
# -------------------
bounds_long_only = tuple((0, 1) for _ in range(n_assets))   # 不允许做空
bounds_long_short = tuple((-1, 1) for _ in range(n_assets)) # 允许做空

vols1, rets1 = efficient_frontier(mean_returns, cov_matrix, bounds_long_only)
vols2, rets2 = efficient_frontier(mean_returns, cov_matrix, bounds_long_short)

# -------------------
# Step 5: 绘图
# -------------------
plt.figure(figsize=(8,6))
plt.plot(vols1, rets1, label="有效前沿（仅多头，0~1)", color="blue")
plt.plot(vols2, rets2, label="有效前沿（多空，-1~1)", color="red", linestyle="--")

plt.xlabel("波动率 (Volatility)")
plt.ylabel("收益率 (Return)")
plt.title("有效前沿对比：不允许做空 vs 允许做空")
plt.legend()
plt.grid(True)
plt.show()
```

7.3 有效前沿解读

运行效果：

- 蓝色曲线：只能买，权重在 (0,1)
- 红色虚线：可以买也能卖，权重在 (-1,1)
- 红色曲线通常比蓝色更"靠左上"，代表更优的风险收益组合

为什么允许做空时，曲线更"靠左上"?

因为允许做空给了优化器更大的"自由度"：

- 只能买 (0~1) → 只能在多头组合里寻找最优点，选择有限
- 可以买也能卖 (-1~1) → 可以同时做多 + 做空，利用对冲手段降低整体波动率，同时提升组合预期收益

但是我们的代码跑出来的结果可能是 蓝色（仅多头）曲线更靠左上，这说明一个关键点：

原因在于：输入的数据 (均值、协方差矩阵) 决定了结果，在设定的 mean_returns 和 cov_matrix 下，：

- 可能“做空”某些股票并不能降低风险，反而增加了组合波动率；
- 或者做空压低了收益率 → 红色曲线就不如蓝色曲线靠左上

正常情况下为什么说“允许做空更优”?

因为在现实市场：

- 股票之间往往有负相关/低相关性，做空一部分能对冲风险。
- 优化器就能构造出“更低风险 + 更高收益”的点 → 曲线更靠左上。

但是！

- 如果所有股票之间 强正相关，做空反而帮不上忙；
- 如果收益率输入本身让某只股票预期“低收益+低风险”，优化器可能更倾向纯多头；
- 这时允许做空就不会改善结果，甚至会让曲线变差。

我们的代码为什么会出现“蓝色更优”?

- 你设置的 mean_returns = [0.12, 0.10, 0.08, 0.07] 全部是正收益。
- cov_matrix 里相关性都比较高（大部分是正数）。
- 这种情况下，做空任何一只股票其实是“减少收益 + 增加风险”，所以曲线变差。
- 可以修改参数，给某只股票一个 负收益，或让两只股票相关性为负：mean_returns = np.array([0.12, 0.10, -0.05, 0.07])

再跑一遍，你就会发现：

- 蓝色曲线还是限制在正权重范围
- 红色曲线能通过“做空股票3”降低整体风险，同时提高收益 → 就会更靠左上

7.4 直观例子

假设：

- 股票 A：收益高，风险也高
- 股票 B：收益低，但和 A 负相关（对冲效果好）

只能买 (0~1)：你最多只能在 A 和 B 之间分配正权重

- 比如 70% A + 30% B
- 风险降低了一点，但效果有限

可以买也能卖 (-1~1)：优化器可能选择：

- 120% A（做多） + -20% B（做空）
- A 的高收益被放大，同时用 B 的做空来对冲风险
- 结果：更高收益、更低波动率 → 曲线左上移

7.5 现实中的问题

虽然数学上"允许做空"会让有效前沿更优，但现实有很多限制：

- 做空需要融资融券，有成本和门槛
- 做空风险大（价格上涨 → 潜在亏损无限）
- 有些市场/账户根本不允许做空

所以在实盘里：

- 股票基金：大多只能做多，不能做空 → 用蓝色曲线
- 对冲基金 / 私募：可以做多做空 → 才能用红色曲线

总结一句：

- 红色曲线更靠左上，是因为"多空对冲"给了优化器更大的操作空间 → 风险/收益比更优
- 但现实中能不能用，要看市场规则和交易者的风险承受能力

八、最优夏普组合点（Tangency Portfolio）

8.1 基本概念

夏普比率 (Sharpe Ratio)：

$$Sharpe = \frac{E[R_p] - R_f}{\sigma_p}$$

Tangency Portfolio（切点组合）：在有效前沿上，找到夏普比率最高的那个点，就是切点组合。

之所以叫 Tangency，是因为在理论里，资本市场线 (CML) 会与有效前沿相切，这个切点就是最优夏普组合。

它代表：在给定资产池里，单位风险能换到最多收益的组合。

> ** 深入学习：** 关于切点组合的数学求解方法、资本市场线（CML）的完整推导、以及无风险资产引入后的投资组合理论扩展，请参考本系列第10天文章《资产配置与均值-方差理论》。

8.2 为什么有"组合点"?

因为：

- 单个股票有自己的风险收益比
- 但不同股票的相关性（协方差）可以用来分散风险
- 多只股票按权重组合，就能找到比单个股票更优的点（风险更低或收益更高）
- 这些点连起来，就形成有效前沿曲线
- 而曲线上夏普比率最高的那个点，就是我们最想要的最优组合

8.3 有效前沿 + 最优夏普组合点代码

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# -------------------
# Step 1: 模拟数据
# -------------------
np.random.seed(42)
n_assets = 4
mean_returns = np.array([0.12, 0.10, 0.08, 0.07])
cov_matrix = np.array([
    [0.04, 0.02, 0.01, 0.00],
    [0.02, 0.03, 0.01, 0.00],
    [0.01, 0.01, 0.02, 0.01],
    [0.00, 0.00, 0.01, 0.015]
])

# -------------------
# Step 2: 函数定义
# -------------------
def portfolio_performance(weights, mean_returns, cov_matrix):
    ret = np.dot(weights, mean_returns)
    vol = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
    return ret, vol

def neg_sharpe(weights, mean_returns, cov_matrix):
    ret, vol = portfolio_performance(weights, mean_returns, cov_matrix)
    return -(ret / vol)

def max_sharpe_portfolio(mean_returns, cov_matrix, bounds):
    n = len(mean_returns)
    init_guess = np.ones(n) / n
    constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}
    result = minimize(
        neg_sharpe, init_guess,
        args=(mean_returns, cov_matrix),
        method="SLSQP",
        bounds=bounds,
        constraints=constraints
    )
    return result

def efficient_frontier(mean_returns, cov_matrix, bounds, points=50):
    target_returns = np.linspace(min(mean_returns), max(mean_returns), points)
    vols, rets = [], []
    for r in target_returns:
        constraints = [
            {"type": "eq", "fun": lambda w: np.sum(w) - 1},
            {"type": "eq", "fun": lambda w: np.dot(w, mean_returns) - r}
        ]
```

```
        result = minimize(
            lambda w: np.sqrt(np.dot(w.T, np.dot(cov_matrix, w))),
            np.ones(len(mean_returns)) / len(mean_returns),
            method="SLSQP",
            bounds=bounds,
            constraints=constraints
        )
        if result.success:
            ret, vol = portfolio_performance(result.x, mean_returns, cov_matrix)
            rets.append(ret)
            vols.append(vol)
    return np.array(vols), np.array(rets)

# -------------------
# Step 3: 计算两种情况
# -------------------
bounds_long_only = tuple((0, 1) for _ in range(n_assets))
bounds_long_short = tuple((-1, 1) for _ in range(n_assets))

vols1, rets1 = efficient_frontier(mean_returns, cov_matrix, bounds_long_only)
vols2, rets2 = efficient_frontier(mean_returns, cov_matrix, bounds_long_short)

# 最优夏普组合点
opt1 = max_sharpe_portfolio(mean_returns, cov_matrix, bounds_long_only)
opt2 = max_sharpe_portfolio(mean_returns, cov_matrix, bounds_long_short)

ret1, vol1 = portfolio_performance(opt1.x, mean_returns, cov_matrix)
ret2, vol2 = portfolio_performance(opt2.x, mean_returns, cov_matrix)

# -------------------
# Step 4: 绘图
# -------------------
plt.figure(figsize=(8,6))
plt.plot(vols1, rets1, label="有效前沿（仅多头）", color="blue")
plt.plot(vols2, rets2, label="有效前沿（多空）", color="red", linestyle="--")

plt.scatter(vol1, ret1, marker="*", color="blue", s=200, label="最优夏普（仅多头）")
plt.scatter(vol2, ret2, marker="*", color="red", s=200, label="最优夏普（多空）")

plt.xlabel("波动率（Volatility）")
plt.ylabel("收益率（Return）")
plt.title("有效前沿 & 最优夏普组合点")
plt.legend()
plt.grid(True)
plt.show()
```

8.4 运行结果解读

图里会有：

- 蓝色曲线：只做多（0~1）

- 红色曲线：可做多空
- 蓝色 ⭐：只能买的最优夏普组合
- 红色 ⭐：多空允许的最优夏普组合

九、为什么切点组合在中间?

9.1 有效前沿的形状回顾

- 有效前沿是一条凸起的曲线（像一条"弯月"）
- 左边 → 风险最低（可能收益也很低）
- 右边 → 收益最高（但风险极大）
- 中间 → 风险和收益的权衡相对合理

9.2 夏普比率的定义

$$Sharpe = \frac{E[R_p] - R_f}{\sigma_p}$$

- 分子：超额收益（收益越高越好）
- 分母：波动率（风险越低越好）
- 目标：单位风险下收益最大

9.3 为什么切点组合在中间?

- **在左端（极低风险组合）**
  - 收益也很低，分子接近 0
  - Sharpe 比率不高
  - 就像存银行，虽然风险低，但收益几乎没有
- **在右端（极高收益组合）**
  - 收益虽然高，但风险飙升，分母太大
  - Sharpe 反而下降
  - 就像满仓买高波动股票，收益不稳定，风险抵消了好处
- **在中间某个点**
  - 既能利用资产间的分散效应，风险没那么高
  - 又能保持相对不错的收益
  - 分子 / 分母 的比例最大
  - 于是形成了 Sharpe 比率最高的切点组合

9.4 直观比喻

想象你在骑自行车上坡：

- 太靠左（坡脚） → 上升不明显，收益小
- 太靠右（坡顶） → 太陡，容易摔下来（风险太大）
- 中间某点 → 上升速度最快又还能保持平衡
- 这就是切点组合

总结一句：切点组合（Tangency Portfolio）位于有效前沿的"中段"，因为它在风险与收益之间找到了最优的平衡点——单位风险带来的超额收益最多。

十、风险平价（Risk Parity）简介

10.1 基本概念

风险平价是一种组合优化方法，其核心思想是让每个资产对组合总风险的贡献相等。

传统等权重：每只股票权重相等 **风险平价：**每只股票的风险贡献相等

10.2 风险贡献计算

对于组合权重 w，资产 i的风险贡献为：

$$RC_i = w_i \times \frac{\partial \sigma_p}{\partial w_i}$$

其中 $\sigma_p = \sqrt{w^T \Sigma w}$是组合波动率。

10.3 风险平价优化

目标函数：

$$\min_w \sum_{i=1}^{n} \sum_{j=1}^{n} (RC_i - RC_j)^2$$

约束条件：

- $\sum w_i = 1$
- $w_i \geq 0$（可选）

10.4 风险平价 vs 均值-方差优化

方法	目标	优点	缺点
均值-方差优化	最大化夏普比率	理论最优	对输入参数敏感

方法	目标	优点	缺点
风险平价	风险贡献均衡	稳健，不依赖收益预测	可能牺牲收益

十一、实战应用建议

11.1 组合优化在量化策略中的应用

1. **多因子选股后的权重优化**
   - 先用因子模型选出股票池
   - 再用组合优化确定最优权重
2. **风险预算管理**
   - 设定目标波动率
   - 通过优化实现风险控制
3. **动态再平衡**
   - 定期重新优化权重
   - 平衡交易成本与优化收益

11.2 实盘注意事项

1. **参数估计的稳定性**
   - 历史数据估计的协方差矩阵可能不稳定
   - 建议使用滚动窗口或正则化方法
2. **交易成本考虑**
   - 优化后的权重变化会产生交易成本
   - 需要在优化目标中加入换手率约束
3. **流动性约束**
   - 大资金需要考虑市场容量
   - 避免过度集中在少数股票

11.3 改进方向

1. Black-Litterman模型
   - 结合市场均衡和主观观点
   - 减少对历史数据的依赖

2. **鲁棒优化**

- 考虑参数不确定性
- 提高策略的稳健性

3. **机器学习方法**

- 使用神经网络预测协方差矩阵
- 动态调整优化参数

十二、总结与展望

12.1 核心要点回顾

概念	定义	应用价值
夏普比率	单位风险下的超额收益	衡量风险调整后收益
有效前沿	最优风险收益组合的集合	寻找最优投资组合
切点组合	夏普比率最高的组合	理论最优配置
风险平价	风险贡献相等的组合	稳健的配置方法

12.2 实战技巧汇总

技巧	说明	注意事项
参数估计	使用足够长的历史数据	注意数据稳定性
约束设置	根据实际限制设置bounds	考虑监管要求
成本控制	在优化中考虑交易成本	平衡收益与成本
稳健性测试	进行敏感性分析	避免过拟合

12.3 下一步学习方向

1. **高级优化方法**

- Black-Litterman模型
- 鲁棒优化
- 动态优化

2. **风险管理**

- VaR和CVaR
- 压力测试
- 情景分析

3. **实盘部署**

- 交易系统集成
- 实时监控
- 绩效归因

课后练习

练习1：基础组合优化

使用以下数据实现基础组合优化：

```
# 数据
mean_returns = np.array([0.15, 0.12, 0.10, 0.08])
cov_matrix = np.array([
    [0.05, 0.02, 0.01, 0.00],
    [0.02, 0.04, 0.01, 0.00],
    [0.01, 0.01, 0.03, 0.01],
    [0.00, 0.00, 0.01, 0.02]
])

# 要求：
# 1. 计算等权重组合的收益和风险
# 2. 优化得到最大夏普比率组合
# 3. 比较两种组合的表现
```

答案：

```
import numpy as np
import pandas as pd
from scipy.optimize import minimize
import matplotlib.pyplot as plt

# 数据
mean_returns = np.array([0.15, 0.12, 0.10, 0.08])
cov_matrix = np.array([
    [0.05, 0.02, 0.01, 0.00],
    [0.02, 0.04, 0.01, 0.00],
    [0.01, 0.01, 0.03, 0.01],
    [0.00, 0.00, 0.01, 0.02]
])

# 1. 计算等权重组合的收益和风险
n_assets = len(mean_returns)
equal_weights = np.ones(n_assets) / n_assets

# 等权重组合收益
```

```python
equal_return = np.dot(equal_weights, mean_returns)

# 等权重组合风险
equal_risk = np.sqrt(np.dot(equal_weights.T, np.dot(cov_matrix, equal_weights)))

# 等权重组合夏普比率
equal_sharpe = equal_return / equal_risk

print("=== 等权重组合表现 ===")
print(f"权重: {equal_weights}")
print(f"预期收益: {equal_return:.4f}")
print(f"风险: {equal_risk:.4f}")
print(f"夏普比率: {equal_sharpe:.4f}")

# 2. 优化得到最大夏普比率组合
def neg_sharpe(weights, mean_returns, cov_matrix):
    """负夏普比率（用于最小化）"""
    port_return = np.dot(weights, mean_returns)
    port_vol = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
    sharpe = port_return / port_vol
    return -sharpe

# 约束条件：权重和为1
constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}

# 边界条件：不允许做空
bounds = tuple((0, 1) for _ in range(n_assets))

# 初始猜测
initial_guess = np.ones(n_assets) / n_assets

# 优化
result = minimize(neg_sharpe, initial_guess, args=(mean_returns, cov_matrix),
                  method="SLSQP", bounds=bounds, constraints=constraints)

optimal_weights = result.x
optimal_return = np.dot(optimal_weights, mean_returns)
optimal_risk = np.sqrt(np.dot(optimal_weights.T, np.dot(cov_matrix, optimal_weights)))
optimal_sharpe = -result.fun

print("\n=== 最优组合表现 ===")
print(f"权重: {optimal_weights}")
print(f"预期收益: {optimal_return:.4f}")
print(f"风险: {optimal_risk:.4f}")
print(f"夏普比率: {optimal_sharpe:.4f}")

# 3. 比较两种组合的表现
print("\n=== 组合比较 ===")
print(f"收益提升: {optimal_return - equal_return:.4f} ({((optimal_return/equal_return-1)*100)
print(f"风险变化: {optimal_risk - equal_risk:.4f} ({((optimal_risk/equal_risk-1)*100):.2f}%)"
print(f"夏普比率提升: {optimal_sharpe - equal_sharpe:.4f} ({((optimal_sharpe/equal_sharpe-1)*1
```

运行结果示例：

```
=== 等权重组合表现 ===
权重: [0.25 0.25 0.25 0.25]
预期收益: 0.1125
风险: 0.1581
夏普比率: 0.7116

=== 最优组合表现 ===
权重: [0.6 0.4 0.0 0.0]
预期收益: 0.1380
风险: 0.1732
夏普比率: 0.7968

=== 组合比较 ===
收益提升: 0.0255 (22.67%)
风险变化: 0.0151 (9.55%)
夏普比率提升: 0.0852 (11.98%)
```

分析：优化后的组合通过集中投资于高收益资产（资产1和2），在适度增加风险的情况下显著提升了收益和夏普比率。

练习2：有效前沿分析

绘制上述数据的有效前沿，并标出：

- 最小方差组合
- 最大夏普比率组合
- 等权重组合

答案：

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# 使用练习1的数据
mean_returns = np.array([0.15, 0.12, 0.10, 0.08])
cov_matrix = np.array([
    [0.05, 0.02, 0.01, 0.00],
    [0.02, 0.04, 0.01, 0.00],
    [0.01, 0.01, 0.03, 0.01],
    [0.00, 0.00, 0.01, 0.02]
])

n_assets = len(mean_returns)

# 1. 最小方差组合
```

```python
def min_variance_portfolio():
    """计算最小方差组合"""
    ones = np.ones(n_assets)
    cov_inv = np.linalg.inv(cov_matrix)

    # 最小方差组合权重
    w_mvp = cov_inv @ ones / (ones.T @ cov_inv @ ones)

    # 计算收益和风险
    ret_mvp = np.dot(w_mvp, mean_returns)
    risk_mvp = np.sqrt(np.dot(w_mvp.T, np.dot(cov_matrix, w_mvp)))

    return w_mvp, ret_mvp, risk_mvp

# 2. 有效前沿上的点
def efficient_frontier_point(target_return):
    """计算给定收益目标下的最小风险组合"""
    ones = np.ones(n_assets)
    cov_inv = np.linalg.inv(cov_matrix)

    # 计算拉格朗日乘数
    A = ones.T @ cov_inv @ ones
    B = ones.T @ cov_inv @ mean_returns
    C = mean_returns.T @ cov_inv @ mean_returns
    D = A * C - B**2

    lambda1 = (C - B * target_return) / D
    lambda2 = (A * target_return - B) / D

    # 最优权重
    w = lambda1 * (cov_inv @ ones) + lambda2 * (cov_inv @ mean_returns)

    # 计算风险
    risk = np.sqrt(np.dot(w.T, np.dot(cov_matrix, w)))

    return w, target_return, risk

# 3. 计算各组合
w_mvp, ret_mvp, risk_mvp = min_variance_portfolio()

# 等权重组合
w_equal = np.ones(n_assets) / n_assets
ret_equal = np.dot(w_equal, mean_returns)
risk_equal = np.sqrt(np.dot(w_equal.T, np.dot(cov_matrix, w_equal)))

# 最大夏普比率组合（使用练习1的结果）
w_optimal = np.array([0.6, 0.4, 0.0, 0.0])
ret_optimal = np.dot(w_optimal, mean_returns)
risk_optimal = np.sqrt(np.dot(w_optimal.T, np.dot(cov_matrix, w_optimal)))

# 4. 生成有效前沿
target_returns = np.linspace(ret_mvp, mean_returns.max() * 1.1, 50)
ef_risks = []
ef_returns = []

for target_ret in target_returns:
```

```
    try:
        _, ret, risk = efficient_frontier_point(target_ret)
        ef_risks.append(risk)
        ef_returns.append(ret)
    except:
        continue

# 5. 绘制图形
plt.figure(figsize=(10, 6))

# 有效前沿
plt.plot(ef_risks, ef_returns, 'b-', linewidth=2, label='有效前沿')

# 关键组合点
plt.scatter(risk_mvp, ret_mvp, color='red', s=100, marker='o', label='最小方差组合')
plt.scatter(risk_equal, ret_equal, color='green', s=100, marker='s', label='等权重组合')
plt.scatter(risk_optimal, ret_optimal, color='orange', s=100, marker='*', label='最大夏普比率组

# 标注
plt.annotate('MVP', (risk_mvp, ret_mvp), xytext=(10, 10),
             textcoords='offset points', fontsize=10)
plt.annotate('Equal Weight', (risk_equal, ret_equal), xytext=(10, -15),
             textcoords='offset points', fontsize=10)
plt.annotate('Max Sharpe', (risk_optimal, ret_optimal), xytext=(10, 10),
             textcoords='offset points', fontsize=10)

plt.xlabel('风险（标准差）')
plt.ylabel('预期收益')
plt.title('有效前沿与关键组合点')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 6. 输出数值结果
print("=== 关键组合对比 ===")
print(f"最小方差组合：收益={ret_mvp:.4f}，风险={risk_mvp:.4f}，夏普={ret_mvp/risk_mvp:.4f}")
print(f"等权重组合：收益={ret_equal:.4f}，风险={risk_equal:.4f}，夏普={ret_equal/risk_equal:.4f
print(f"最大夏普组合：收益={ret_optimal:.4f}，风险={risk_optimal:.4f}，夏普={ret_optimal/risk_op
```

分析：图形显示有效前沿是一条向上凸的曲线，最小方差组合位于最左侧，最大夏普比率组合位于切点位置，等权重组合通常位于有效前沿内部。

练习3：敏感性分析

分析协方差矩阵估计误差对优化结果的影响：

- 在原始协方差矩阵基础上加入随机噪声
- 比较优化权重的变化
- 评估策略的稳健性

答案：

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# 原始数据
mean_returns = np.array([0.15, 0.12, 0.10, 0.08])
cov_matrix_original = np.array([
    [0.05, 0.02, 0.01, 0.00],
    [0.02, 0.04, 0.01, 0.00],
    [0.01, 0.01, 0.03, 0.01],
    [0.00, 0.00, 0.01, 0.02]
])

n_assets = len(mean_returns)

def optimize_portfolio(mean_returns, cov_matrix):
    """优化组合权重"""
    def neg_sharpe(weights, mean_returns, cov_matrix):
        port_return = np.dot(weights, mean_returns)
        port_vol = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
        return -(port_return / port_vol)

    constraints = {"type": "eq", "fun": lambda w: np.sum(w) - 1}
    bounds = tuple((0, 1) for _ in range(n_assets))
    initial_guess = np.ones(n_assets) / n_assets

    result = minimize(neg_sharpe, initial_guess, args=(mean_returns, cov_matrix),
                      method="SLSQP", bounds=bounds, constraints=constraints)

    return result.x

# 1. 原始最优组合
weights_original = optimize_portfolio(mean_returns, cov_matrix_original)
print("=== 原始最优权重 ===")
print(f"权重: {weights_original}")

# 2. 敏感性分析
np.random.seed(42)  # 固定随机种子
n_simulations = 100
noise_levels = [0.01, 0.05, 0.10, 0.20]  # 不同的噪声水平

results = {}

for noise_level in noise_levels:
    weights_list = []
```

```
    returns_list = []
    risks_list = []
    sharpes_list = []

    for _ in range(n_simulations):
        # 添加随机噪声到协方差矩阵
        noise = np.random.normal(0, noise_level, cov_matrix_original.shape)
        # 确保对称性
        noise = (noise + noise.T) / 2
        cov_matrix_noisy = cov_matrix_original + noise

        # 确保正定性（如果必要）
        eigenvals = np.linalg.eigvals(cov_matrix_noisy)
        if np.min(eigenvals) <= 0:
            cov_matrix_noisy = cov_matrix_original + noise * 0.5

        # 优化组合
        try:
            weights = optimize_portfolio(mean_returns, cov_matrix_noisy)
            weights_list.append(weights)

            # 计算组合表现
            ret = np.dot(weights, mean_returns)
            risk = np.sqrt(np.dot(weights.T, np.dot(cov_matrix_noisy, weights)))
            sharpe = ret / risk

            returns_list.append(ret)
            risks_list.append(risk)
            sharpes_list.append(sharpe)
        except:
            continue

    results[noise_level] = {
        'weights': np.array(weights_list),
        'returns': np.array(returns_list),
        'risks': np.array(risks_list),
        'sharpes': np.array(sharpes_list)
    }

# 3. 分析结果
print("\n=== 敏感性分析结果 ===")
for noise_level in noise_levels:
    data = results[noise_level]

    print(f"\n噪声水平: {noise_level*100:.0f}%")
    print(f"权重均值: {np.mean(data['weights'], axis=0)}")
    print(f"权重标准差: {np.std(data['weights'], axis=0)}")
    print(f"收益均值±标准差: {np.mean(data['returns']):.4f}±{np.std(data['returns']):.4f}")
    print(f"风险均值±标准差: {np.mean(data['risks']):.4f}±{np.std(data['risks']):.4f}")
    print(f"夏普比率均值±标准差: {np.mean(data['sharpes']):.4f}±{np.std(data['sharpes']):.4f}")

# 4. 可视化权重变化
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

```
axes = axes.flatten()

for i, noise_level in enumerate(noise_levels):
    data = results[noise_level]
    weights_data = data['weights']

    # 绘制权重分布
    for j in range(n_assets):
        axes[i].hist(weights_data[:, j], alpha=0.6, label=f'资产{j+1}', bins=20)

    axes[i].axvline(weights_original[i] if i < n_assets else 0, color='red',
                    linestyle='--', linewidth=2, label='原始权重')
    axes[i].set_title(f'噪声水平: {noise_level*100:.0f}%')
    axes[i].set_xlabel('权重')
    axes[i].set_ylabel('频次')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 5. 稳健性评估
print("\n=== 稳健性评估 ===")
for noise_level in noise_levels:
    data = results[noise_level]

    # 计算权重与原始权重的平均偏差
    weight_deviations = np.mean(np.abs(data['weights'] - weights_original), axis=0)
    avg_deviation = np.mean(weight_deviations)

    # 计算夏普比率的变异系数
    sharpe_cv = np.std(data['sharpes']) / np.mean(data['sharpes'])

    print(f"噪声{noise_level*100:.0f}%: 平均权重偏差={avg_deviation:.4f}, 夏普比率变异系数={sharp

# 6. 建议
print("\n=== 稳健性改进建议 ===")
print("1. 使用更长的历史数据估计协方差矩阵")
print("2. 采用正则化方法（如Ledoit-Wolf收缩估计）")
print("3. 使用滚动窗口更新协方差矩阵")
print("4. 考虑使用Black-Litterman模型结合主观观点")
print("5. 实施权重约束，避免过度集中")
```

分析结果：

- 随着噪声水平增加，优化权重的波动性显著增大
- 高噪声情况下，组合表现变得不稳定
- 建议采用正则化方法和更稳健的估计技术

关键发现：

1. **权重敏感性**：协方差矩阵的小幅变化可能导致权重的大幅调整
2. **稳健性不足**：传统均值-方差优化对参数估计误差敏感
3. **改进方向**：需要采用更稳健的优化方法和参数估计技术

总结：组合优化是现代投资组合理论的核心，通过数学优化方法寻找最优的资产配置。夏普比率、有效前沿和切点组合是理解组合优化的关键概念。在实际应用中，需要平衡理论最优与现实约束，考虑参数估计的不确定性和交易成本的影响。掌握这些方法将为构建稳健的量化投资策略奠定坚实基础。

量化交易入门指南 · 目录 ≡

<上一篇
量化交易入门指南：第20天 – 风格因子控制（行业/市值）——风险中性化

下一篇 >
量化交易入门指南：第22天 – Barra 风格因子简介

内容含AI生成图片，注意甄别

阅读 221

广告
Today

疯狂水世界
一块木筏，就是末日里的全部希望
现已推出
疯狂水世界！启动！
船长在拾荒
进入小游戏